

## Lecture 1- Introduction to AI & Machine Learning

### Key points:

- Intelligence is the **ability to solve problems and adapt to situations**.
- **Artificial Intelligence (AI)** aims to create systems that mimic human cognitive functions.
- AI includes several fields such as **Machine Learning, Robotics, NLP, and Computer Vision**.
- **Machine Learning (ML)** is a subset of AI that allows systems to **learn from data instead of explicit programming**.
- ML algorithms **discover patterns and relationships in data** to make predictions or decisions.
- Learning is defined by **Experience (E), Task (T), and Performance measure (P)**.
- **Data Science** involves collecting, cleaning, analyzing, and extracting insights from data.
- **Features** are input variables used to make predictions.
- **Target/Label** is the output the model predicts.
- Machine learning models aim to **map input features to outputs**.

### Types of Machine Learning

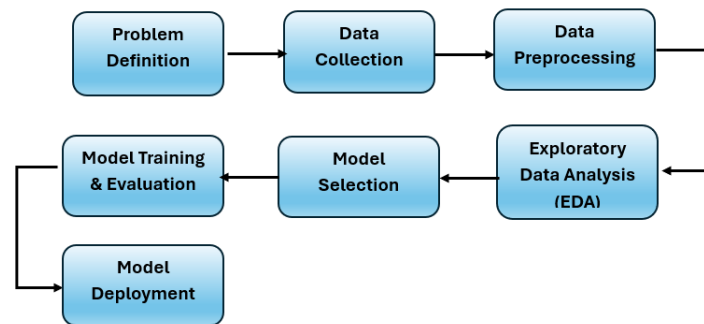
- **Supervised Learning:** uses labeled input-output data.
  - ✓ **Classification:** predicts categorical outputs (e.g., spam detection).
  - ✓ **Regression:** predicts continuous values (e.g., house price).
- **Unsupervised Learning:** works with unlabeled data to discover patterns.
  - ✓ **Clustering:** groups similar data points.
  - ✓ **Dimensionality Reduction:** reduces number of variables.
  - ✓ **Anomaly Detection:** identifies unusual data points.
  - ✓ **Association Rules:** finds relationships between items in datasets.
- **Reinforcement Learning:** agent learns through interaction with an environment using rewards.

### Model Evaluation

- Used to **measure performance, compare models, and select the best algorithm**.

## Lecture 2: Machine Learning Workflow & Pipeline

A machine learning project follows a **structured pipeline** consisting of several stages to transform raw data into a deployable model.



**Data Preparation:** Transforms raw data into a **clean and usable format**.

Includes:

- Data preprocessing
- Feature engineering

**Data Preprocessing:** Process of **cleaning and transforming raw data** to make it suitable for machine learning.

Common tasks:

- Data cleaning
- Handling missing values
- Handling outliers
- Data splitting
- Feature scaling
- Data transformation

Difference:

- **Noise:** random errors
- **Outliers:** extreme abnormal values

**Feature Scaling:** Ensures all features have **similar numeric ranges**.

Important because: Features may have different scales (e.g., age vs salary).

Common techniques: Normalization Methods & Standardization Methods

**Data Transformation:** Converting data into consistent formats.

Example:

- Converting categorical variables into numerical form.

Techniques:

- One-hot encoding
- Label encoding

**Feature Engineering:** Creating or transforming features to **improve model performance**.

Techniques:

- Feature creation: Create new features from existing)
- Feature encoding: Transform categorical variables into numeric values.  
Methods:
  - ✓ Label encoding
  - ✓ Ordinal encoding
  - ✓ One-hot encoding
  - ✓ Dummy variable encoding
  - ✓ Target encoding
- Feature binning (Discretization): Convert continuous variables into discrete categories.  
Types:
  - ✓ Equal width binning
  - ✓ Equal depth (frequency) binning
  - ✓ Quantile binning

### ✓ Core Idea of Lecture 2

Machine learning follows a **structured pipeline that converts raw data into a deployed predictive model through preprocessing, analysis, modeling, evaluation, and deployment.**

## Lecture 3: Supervised ML (Regression Models)

### Key points:

- **Supervised learning** maps input variables to output variables using labeled data.
- A supervised model represents a **mathematical function** mapping inputs  $x$  to outputs  $y$ .
- **Inference** is computing predictions from inputs using the trained model.
- Models contain **parameters** that control the relationship between inputs and outputs.
- **Training** means finding parameters that best predict outputs using a training dataset of input–output pairs.
- The quality of predictions is measured using a **loss function** (cost function).
- The goal of training is to **minimize the loss function**.

### Types of Supervised Learning

- **Classification:** predicts discrete categories (e.g., spam detection).
- **Regression:** predicts continuous values (e.g., house price prediction).

### Regression in Machine Learning

- Regression estimates the **relationship between features (inputs) and a continuous target variable**.
- Used for predictions such as:
  - House prices
  - Market trends
  - Economic forecasting
  - Correlation analysis between variables.

### Types of Regression

- **Simple Linear Regression:** one independent variable predicts one dependent variable.
- **Multiple Linear Regression:** multiple independent variables predict one dependent variable.
- **Polynomial Regression:** models nonlinear relationships using polynomial functions.

### Linear Regression Model

The regression equation:  $y = \beta_0 + \beta_1 x + \epsilon$

- $\beta_0$ : intercept
- $\beta_1$ : slope (rate of change of  $y$  with respect to  $x$ )
- $\epsilon$ : random error term.

### Residual / Prediction Error

- Residual = difference between **actual value** and **predicted value**.

$$e_i = y_i - \hat{y}_i$$

## Least Squares Method

- Used to find the **best-fitting regression line**.
- Minimizes the **sum of squared errors (Residual Sum of Squares – RSS)**.

$$RSS = \sum (y_i - \hat{y}_i)^2$$

## Model Training and Testing

- **Training:** adjust parameters to minimize loss.
- **Testing:** evaluate the model using unseen data to check **generalization ability**.

Possible problems:

- **Underfitting:** model too simple.
- **Overfitting:** model too complex.

## Model Evaluation Metrics

Used to measure regression performance:

- **SST (Total Sum of Squares):** variation of actual values from their mean.
- **SSE (Sum of Squared Error):** difference between actual and predicted values.
- **SSR (Sum of Squared Regression):** variation explained by the model.

$$\text{Relationship: } SST = SSR + SSE$$

## Important Evaluation Metrics

- **R<sup>2</sup> (Coefficient of Determination):** measures how well the model explains variance in the data.
- **RMSE (Root Mean Squared Error):** measures prediction error magnitude.
- **F-test:** evaluates significance of regression model.

## Loss vs Cost Function

- **Loss function:** error for a single training sample.
- **Cost function:** average loss across all samples (used for optimization).

## Lecture 4 – Regularization, Bias, Variance & Multicollinearity

### Correlation

- **Correlation** measures the relationship between two variables.
- It indicates whether variables **move together** or **in opposite directions**.
- **Important:** Correlation **does NOT** imply causation.

### Correlation Types

- **Positive correlation:** both variables increase or decrease together.
- **Negative correlation:** one increases while the other decreases.
- **No correlation:** variables have no linear relationship.

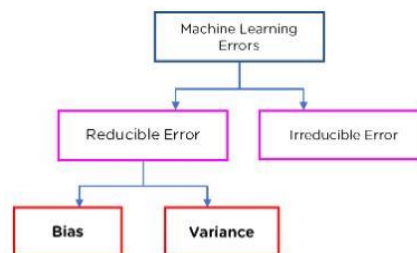
### Correlation Coefficient (r)

- Measures **strength of linear relationship** between variables.
- Represented by **Pearson correlation coefficient (r)**.
- Range: **-1 to +1**.

| Value of r | Meaning                      |
|------------|------------------------------|
| +1         | Perfect positive correlation |
| 0          | No correlation               |
| -1         | Perfect negative correlation |

**Multicollinearity:** Occurs when **independent variables are highly correlated with each other**.

**Machine Learning Model Error:** it measures **how accurate predictions are**.



| Concept     | Definition                                   | Characteristics                                            | When High          | When Low                        | How to Reduce             |
|-------------|----------------------------------------------|------------------------------------------------------------|--------------------|---------------------------------|---------------------------|
| <b>Bias</b> | Difference between the average prediction of | Indicates how much the model assumptions simplify the true | • Model too simple | • Model fits training data well | • Use more complex models |

| Concept         | Definition                                                                     | Characteristics                                                              | When High                                                                                                                                        | When Low                                                                                                            | How to Reduce                                                                                                                                                                     |
|-----------------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                 | the model and the true value.                                                  | relationship in the data.                                                    | <ul style="list-style-type: none"> <li>• Cannot capture real patterns</li> <li>• Leads to underfitting</li> </ul>                                | <ul style="list-style-type: none"> <li>• Captures underlying patterns better</li> </ul>                             | <ul style="list-style-type: none"> <li>• Increase number of features</li> <li>• Increase training data</li> <li>• Reduce regularization</li> </ul>                                |
| <b>Variance</b> | Measures how much the model predictions change when the training data changes. | Indicates the sensitivity of the model to fluctuations in the training data. | <ul style="list-style-type: none"> <li>• Model too sensitive to training data</li> <li>• Learns noise</li> <li>• Leads to overfitting</li> </ul> | <ul style="list-style-type: none"> <li>• Stable predictions</li> <li>• Better generalization to new data</li> </ul> | <ul style="list-style-type: none"> <li>• Cross-validation</li> <li>• Feature selection</li> <li>• Regularization</li> <li>• Ensemble methods</li> <li>• Early stopping</li> </ul> |

There is a **trade-off between bias and variance**.

| Case                      | Result       |
|---------------------------|--------------|
| High bias + Low variance  | Underfitting |
| Low bias + High variance  | Overfitting  |
| Low bias + Low variance   | Ideal model  |
| High bias + High variance | Poor model   |

Goal: **balance both for best generalization**.

### Underfitting vs Overfitting

| Underfitting               | Overfitting                             |
|----------------------------|-----------------------------------------|
| Model too simple           | Model too complex                       |
| High bias                  | High variance                           |
| Poor train & test accuracy | Good training but poor test performance |

**Regularization:** it reduces **overfitting** by **adding penalties to model coefficients**.

Cost Function:

$$Cost = Loss + Regularization T$$

*erm*

It forces the model to stay **simpler and more stable**.

| Technique                        | Penalty Type                                             | Main Idea                                               | Key Effects                                                                                                                                                                    | Best Used When                                                             |
|----------------------------------|----------------------------------------------------------|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>L1 Regularization (Lasso)</b> | Adds absolute value of coefficients to the loss function | Forces some coefficients to shrink to exactly zero      | <ul style="list-style-type: none"> <li>• Performs automatic feature selection</li> <li>• Removes irrelevant features</li> <li>• Produces a simpler model</li> </ul>            | When datasets have many features and some are irrelevant                   |
| <b>L2 Regularization (Ridge)</b> | Adds squared value of coefficients to the loss function  | Shrinks coefficients toward zero but never exactly zero | <ul style="list-style-type: none"> <li>• Reduces large coefficient values</li> <li>• Keeps all features in the model</li> <li>• Helps reduce multicollinearity</li> </ul>      | When features are correlated and you want to keep all variables            |
| <b>Elastic Net</b>               | Combines L1 + L2 penalties                               | Uses both feature selection and coefficient shrinking   | <ul style="list-style-type: none"> <li>• Performs feature selection (L1)</li> <li>• Maintains stable coefficients (L2)</li> <li>• Balances simplicity and stability</li> </ul> | When features are highly correlated and L1 alone removes too many features |

**Effect of  $\lambda$  (Lambda):**  $\lambda$  controls **regularization strength** → Higher  $\lambda$  → **higher bias, lower variance**.

| Lambda Value         | Effect                            |
|----------------------|-----------------------------------|
| Small $\lambda$      | Model $\approx$ normal regression |
| Medium $\lambda$     | Some coefficients shrink          |
| Very large $\lambda$ | All coefficients $\rightarrow 0$  |



## Lecture 5 - Supervised ML (Classification Part 1 logistic+KNN)

### Types of Classification Problems

| Type                              | Description                                 | Examples                                 |
|-----------------------------------|---------------------------------------------|------------------------------------------|
| <b>Binary Classification</b>      | Two possible classes                        | Spam detection, fraud detection          |
| <b>Multi-class Classification</b> | More than two classes                       | Image classification, sentiment analysis |
| <b>Multi-label Classification</b> | One instance can belong to multiple classes | Image tagging, medical diagnosis         |

### Logistic Regression

- A statistical model used for binary classification.
- Predicts the **probability that an instance belongs to a class**.

#### Key idea

Uses the **logistic (sigmoid) function**:

- Output values are between **0 and 1**
- Represents **probability of class membership**

Example:

- Probability patient has disease = **0.82**

### Logistic vs Linear Regression

| Linear Regression                 | Logistic Regression                          |
|-----------------------------------|----------------------------------------------|
| Used for <b>continuous output</b> | Used for <b>categorical output</b>           |
| Example: predicting price         | Example: predicting spam/not spam            |
| Output can be any real value      | Output is probability between <b>0 and 1</b> |

### Maximum Likelihood Estimation (MLE)

- Used to **estimate parameters** of logistic regression.
- It finds parameters that **maximize probability of observing the data**.

Important relation: **MLE** ↔ **minimizing cross-entropy (log loss)**

### K-Nearest Neighbors (KNN)

- A **supervised learning algorithm** that classifies data based on **nearest neighbors**.

Key assumption: **Similar data points are close to each other**

### How KNN Works

1. Choose **K** (number of neighbors).
2. Calculate **distance** between new point and training data.
3. Sort distances.
4. Select **K nearest neighbors**.
5. Apply **majority voting** (classification).

For regression: Prediction = **average value of neighbors**

### Choosing K in KNN

- **Small K**
  - Sensitive to noise
  - High variance
- **Large K**
  - Smooth predictions
  - Higher bias

Best K is chosen using **cross-validation or error-rate plots**.

### Evaluation criteria: Confusion Matrix

A table comparing predicted vs actual classes.

| Term | Meaning                       |
|------|-------------------------------|
| TP   | Correct positive prediction   |
| TN   | Correct negative prediction   |
| FP   | Incorrect positive prediction |
| FN   | Incorrect negative prediction |

**Accuracy:** it measures **how often the model predicts correctly**.

$$\text{Accuracy} = (\text{Correct predictions}) / (\text{Total predictions})$$

### Precision & Recall

| Metric                      | Meaning                                                |
|-----------------------------|--------------------------------------------------------|
| <b>Precision</b>            | Correct positive predictions among predicted positives |
| <b>Recall (Sensitivity)</b> | Ability to find all actual positives                   |

## F1-Score

- Harmonic mean of **precision and recall**.
- Used when both metrics are important.

High F1 score = **balanced performance**.

**Specificity (True Negative Rate):** it measures how well the model identifies negative cases.

Formula concept:  $TN / (TN + FP)$

## ROC Curve & AUC

### ROC Curve

- Plots:
  - **TPR (True Positive Rate)** vs
  - **FPR (False Positive Rate)**

**AUC (Area Under Curve)** measures classifier quality.

| AUC | Meaning            |
|-----|--------------------|
| 1   | Perfect classifier |
| 0.5 | Random guess       |
| 0   | Completely wrong   |

## Lecture 6 – Classification (SVM + Decision Trees)

### Support Vector Machine (SVM)

- SVM is a supervised machine learning algorithm used for **classification (and sometimes regression)**.
- It works by finding the **optimal hyperplane that separates classes in the feature space**.

### Important concepts

- **Hyperplane:** decision boundary separating classes.
- **Support Vectors:** data points **closest to the hyperplane** that determine the boundary.
- **Margin:** distance between the hyperplane and the nearest data points.

Goal:

- **Maximize the margin** to improve classification performance.

### Linear SVM

- Works when data is **linearly separable**.
- Decision boundary formula:

$$f(x) = \text{sign}(w^T x + b)$$

Where:

- **w** → weight vector
- **b** → bias term

### Soft Margin SVM

Used when data **cannot be perfectly separated**.

- Introduces **slack variables ( $\xi$ )** to allow some misclassification.
- Uses parameter **C**.

**Role of C:** it controls the trade-off between **Large margin** and **Classification error (misclassified points)**

| C Value | Effect                               |
|---------|--------------------------------------|
| Large C | Less tolerance for misclassification |
| Small C | More tolerance for errors            |

## Non-Linear SVM

When data is **not linearly separable**, SVM maps data to a **higher-dimensional space**. This allows the model to find a **linear separator in the new space**.

### Kernel Trick

A **kernel function** calculates similarity between data points and implicitly maps them into higher dimensions.

Common kernels:

- **Linear kernel**
- **Polynomial kernel**
- **RBF (Radial Basis Function)** – most common

Why RBF?

- Handles non-linear data well
- Fewer parameters
- Numerically stable

**Decision Tree (DT):** A supervised learning algorithm used for **classification and regression**. It represents decisions in a **tree-like structure**.

Structure:

| Component      | Meaning                 |
|----------------|-------------------------|
| Root node      | Entire dataset          |
| Internal nodes | Feature-based decisions |
| Branches       | Outcomes of decision    |
| Leaf nodes     | Final prediction        |

### How Decision Trees Work

Prediction process:

1. Start at the **root node**
2. Check feature condition
3. Follow the branch
4. Continue until reaching a **leaf node**
5. Output the predicted class

**Splitting in Decision Trees:** The algorithm splits data based on the **best attribute** to create **pure subsets** (i.e., Reduce **impurity** of data.)

**Attribute Selection Measures (ASM):** Two main techniques used to choose the best split:

| Method           | Purpose           |
|------------------|-------------------|
| Information Gain | Based on entropy  |
| Gini Index       | Measures impurity |

| Measure                 | Definition                                                                        | Key Idea                                                    | Interpretation / Rule                                                                                                                                                                                     |
|-------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Entropy</b>          | Measures the impurity or randomness in a dataset.                                 | Indicates how mixed the classes are in a node.              | <ul style="list-style-type: none"> <li>• High entropy → classes are highly mixed</li> <li>• Low entropy → dataset is more pure</li> <li>• Decision trees try to reduce entropy after splitting</li> </ul> |
| <b>Information Gain</b> | Measures the reduction in entropy after splitting the dataset using an attribute. | Shows how much information a feature gives about the class. | Formula idea: $IG = Entropy(\text{parent}) - \text{Weighted Entropy}(\text{children})$<br>Rule: choose the attribute with the highest information gain                                                    |
| <b>Gini Index</b>       | Another metric that measures impurity of a node.                                  | Used to evaluate how good a split is.                       | <ul style="list-style-type: none"> <li>• 0 → perfectly pure node</li> <li>• Higher value → more impurity</li> </ul> Best split: attribute with the lowest Gini index                                      |

## Lecture 7 – Ensemble Learning

### Ensemble Learning

- **Ensemble learning** combines multiple machine learning models to create a **stronger predictive model**.
- The idea is that **many weak models together perform better than a single model**.

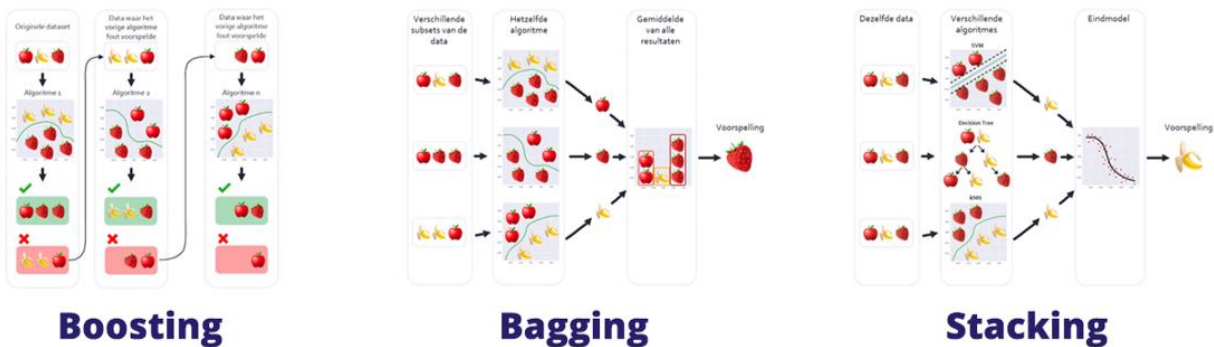
Key benefits:

- Higher **accuracy**
- Better **robustness**
- Reduced **variance and bias**

Ensemble methods combine predictions using:

- **Voting**
- **Averaging**
- **Weighted averaging**

| Method                  | Idea                                             |
|-------------------------|--------------------------------------------------|
| <b>Max Voting</b>       | Class with the highest number of votes is chosen |
| <b>Averaging</b>        | Average prediction of multiple models            |
| <b>Weighted Average</b> | Predictions combined using different weights     |



### Bagging (Bootstrap Aggregating)

- Ensemble technique that **reduces variance**.
- Multiple models are trained on **different bootstrap samples** of the dataset.

### Key idea

- Sampling with replacement
- Train models **in parallel**

Process:

1. Create bootstrap datasets.
2. Train multiple models.
3. Combine predictions by **averaging or voting**.

### Bootstrap Sampling

- Random sampling **with replacement** from the dataset.
- Some data points may appear **multiple times** in a sample.

Purpose: Create different training datasets for ensemble models.

| Method                                          | Description                                                                                              | Key Ideas / Steps                                                                                                                                         | Result / Output                                                                                                         |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Random Forest</b>                            | Ensemble model composed of many decision trees trained on different random subsets of data and features. | <ul style="list-style-type: none"><li>• Wisdom of the crowd</li><li>• Feature randomness</li><li>• Each tree learns from different data samples</li></ul> | Predictions combined using majority voting (classification) or average (regression)                                     |
| <b>Random Subspace Method</b>                   | Similar to bagging but uses random subsets of features instead of samples for training models.           | Steps: 1. Select random subset of features<br>2. Train models using those features<br>3. Aggregate predictions                                            | Improves diversity between models and enhances overall performance                                                      |
| <b>Extra Trees (Extremely Randomized Trees)</b> | Ensemble method similar to Random Forest, but introduces more randomness during tree construction.       | <ul style="list-style-type: none"><li>• Split values are selected randomly instead of searching for the best split</li></ul>                              | <ul style="list-style-type: none"><li>• Higher randomness</li><li>• Faster training compared to Random Forest</li></ul> |

### Boosting

- Ensemble technique that **improves weak learners sequentially**.

Key idea:

- Each new model **focuses on correcting errors of previous models**.

Steps:

1. Train first model.



2. Increase weight for misclassified samples.
3. Train next model.
4. Combine models using **weighted predictions**.

Goal:

- **Reduce bias and improve accuracy.**

| Method                                     | Description                                                                            | Main Idea / Process                                                                                                                                                                           | Key Features / Result                                                                                                                                                                                                                            |
|--------------------------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>AdaBoost (Adaptive Boosting)</b>        | Boosting algorithm that adjusts sample weights during training.                        | Process:<br>1. Train weak classifier<br>2. Increase weight of misclassified samples<br>3. Train next classifier focusing on difficult samples<br>4. Combine classifiers using weighted voting | Improves performance by focusing on hard-to-classify data                                                                                                                                                                                        |
| <b>Gradient Boosting</b>                   | Sequential ensemble algorithm that builds models to correct errors of previous models. | Steps:<br>1. Make initial prediction<br>2. Compute residual errors<br>3. Train new model on residuals<br>4. Update predictions iteratively                                                    | Gradually minimizes prediction errors and improves model accuracy                                                                                                                                                                                |
| <b>XGBoost (Extreme Gradient Boosting)</b> | Optimized and efficient implementation of Gradient Boosting.                           | Builds decision trees sequentially while optimizing the loss function and correcting previous errors.                                                                                         | <ul style="list-style-type: none"> <li>• High computational efficiency</li> <li>• Regularization to prevent overfitting</li> <li>• Handles missing values</li> <li>• Not sensitive to outliers</li> <li>• No feature scaling required</li> </ul> |

### Bagging vs Boosting

| Feature   | Bagging         | Boosting          |
|-----------|-----------------|-------------------|
| Training  | Parallel        | Sequential        |
| Data      | Random sampling | Weighted sampling |
| Main goal | Reduce variance | Reduce bias       |

| Feature | Bagging       | Boosting |
|---------|---------------|----------|
| Example | Random Forest | AdaBoost |

**Stacking (Stacked Generalization):** Ensemble technique that **combines different models using another model.**

Structure:

| Level                        | Description                           |
|------------------------------|---------------------------------------|
| <b>Level-0 (Base Models)</b> | Multiple ML models trained on dataset |
| <b>Level-1 (Meta Model)</b>  | Model that combines predictions       |

Example:

- Base models: SVM, Decision Tree, Random Forest
- Meta model: Logistic Regression

## Lecture 8 – Model Evaluation, Cross-Validation & Hyperparameter Tuning.

### Model Evaluation

- Measures how well a model **generalizes to unseen data**.
- Helps detect **overfitting and underfitting**.

Common metrics:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC
- RMSE

### Cross-validation methods

| Method                                    | Description                                                                                       | Process / Key Idea                                                                                                             | Advantages                                                                                                     | Disadvantages / Notes                                                                 |
|-------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Hold-Out Method</b>                    | Simple model evaluation technique that splits the dataset into training and testing sets.         | 1. Split dataset into Training (70–80%) and Test (20–30%) 2. Train model on training data 3. Evaluate model on test data       | <ul style="list-style-type: none"><li>• Simple</li><li>• Fast to implement</li></ul>                           | <ul style="list-style-type: none"><li>• Results depend on random data split</li></ul> |
| <b>Cross-Validation</b>                   | Statistical method used to evaluate model performance more reliably by repeatedly splitting data. | Model is trained and tested multiple times on different subsets of data.                                                       | <ul style="list-style-type: none"><li>• Reduces bias and variance</li><li>• More reliable evaluation</li></ul> | More computational cost than hold-out                                                 |
| <b>K-Fold Cross-Validation</b>            | Dataset is divided into K equal parts (folds) and evaluated multiple times.                       | 1. Split dataset into K folds<br>2. Train on K–1 folds<br>3. Test on remaining fold<br>4. Repeat K times<br>5. Average results | Provides stable performance estimate                                                                           | Slightly higher computation                                                           |
| <b>Stratified K-Fold Cross-Validation</b> | Improved K-Fold that preserves class distribution in each fold.                                   | Each fold keeps the same class ratio as the full dataset.                                                                      | Best for imbalanced datasets                                                                                   | Not suitable for time-series data                                                     |

| Method                                        | Description                                              | Process / Key Idea                                                                                                                                        | Advantages                                                                                           | Disadvantages / Notes     |
|-----------------------------------------------|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|---------------------------|
| <b>Leave-One-Out Cross-Validation (LOOCV)</b> | Special case of K-Fold where K = number of samples.      | <ul style="list-style-type: none"> <li>• Use one sample as test</li> <li>• Train on <math>n-1</math> samples</li> <li>• Repeat for all samples</li> </ul> | <ul style="list-style-type: none"> <li>• Very accurate evaluation</li> </ul>                         | Computationally expensive |
| <b>Repeated K-Fold Cross-Validation</b>       | Runs K-Fold multiple times with different random splits. | Example: 5-Fold repeated 3 times $\rightarrow$ 15 evaluations                                                                                             | <ul style="list-style-type: none"> <li>• Improves reliability</li> <li>• Reduces variance</li> </ul> | Higher computational cost |

**Hyperparameters:** Parameters **set before training** the model.

| Algorithm             | Hyperparameters                |
|-----------------------|--------------------------------|
| Polynomial Regression | Degree                         |
| Logistic Regression   | Regularization                 |
| Decision Tree         | Max depth                      |
| Random Forest         | Number of features             |
| Neural Network        | Layers, neurons, learning rate |

**Hyperparameter Space:** The set of **all possible hyperparameter combinations**.

Properties:

- Each axis = one hyperparameter
- Each point = unique parameter combination.

Goal:

- Find the **optimal combination**.

### Hyperparameter Tuning Methods

| Method                | Idea                                      |
|-----------------------|-------------------------------------------|
| Manual Search         | Trial-and-error                           |
| Grid Search           | Test all combinations                     |
| Random Search         | Random combinations                       |
| Halving Grid Search   | Eliminate weak combinations early         |
| Halving Random Search | Random search with resource reduction     |
| Bayesian Optimization | Predict best parameters using probability |

## Lecture 9 – Unsupervised Machine Learning (Clustering, PCA, Association Rules).

### Unsupervised Machine Learning

- In **unsupervised learning**, the **correct labels or classes are not known**.
- The goal is to **discover hidden patterns or structures in data**.

Main tasks:

- Clustering
- Dimensionality Reduction
- Association Rule Mining

### Clustering

- **Clustering** groups similar data points into **clusters based on similarity**.

Goal: **High intra-cluster similarity and Low inter-cluster similarity**

Applications:

- Customer segmentation
- Document grouping
- Image analysis
- Market segmentation

### Types of Clustering Approaches

| Approach              | Idea                                    |
|-----------------------|-----------------------------------------|
| <b>Centroid-based</b> | Assign points to nearest cluster center |
| <b>Hierarchical</b>   | Merge or split clusters progressively   |

Common algorithms:

- **K-Means**
- **Hierarchical clustering**

**Distance Measures in Clustering:** determine **similarity between points**.

Common measures:

- **Euclidean distance (L2 norm)**
- **Manhattan distance (L1 norm)**
- **Maximum distance ( $L^\infty$  norm)**

## K-Means Clustering

- A centroid-based clustering algorithm.
- Divides data into **K clusters**.

Goal: Minimize distance between data points and cluster centroid.

### K-Means Algorithm Steps

1. Choose **K cluster centroids** randomly.
2. Assign each data point to the **nearest centroid**.
3. Recalculate centroids (mean of cluster points).
4. Repeat until **cluster assignments stop changing**.

### Determining Optimal Number of Clusters

| Method                   | Description                                                                                    | Key Idea                                                                                                                                 | Interpretation                                                              |
|--------------------------|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>Elbow Method</b>      | Uses Sum of Squared Errors (SSE) to evaluate clustering performance for different values of K. | Plot K vs SSE and observe the curve. The optimal K is at the “elbow point”, where the decrease in SSE starts slowing down significantly. | The elbow point indicates a balance between low error and model simplicity. |
| <b>Silhouette Method</b> | Measures how well each data point fits within its assigned cluster compared to other clusters. | Computes a Silhouette score for each point.                                                                                              | +1: well clustered<br>0: clusters overlap<br>-1: misclassified point        |

**Hierarchical Clustering:** Groups data by **building a hierarchy of clusters**.

| Type                             | Description                                   |
|----------------------------------|-----------------------------------------------|
| <b>Agglomerative (Bottom-Up)</b> | Start with individual points → merge clusters |
| <b>Divisive (Top-Down)</b>       | Start with one cluster → split repeatedly     |

Visualization: **Dendrogram (tree diagram)**

**Linkage Methods in Hierarchical Clustering:** Used to measure distance between clusters.

| Method                  | Meaning                           |
|-------------------------|-----------------------------------|
| <b>Single linkage</b>   | Distance between closest points   |
| <b>Complete linkage</b> | Distance between farthest points  |
| <b>Average linkage</b>  | Average distance between clusters |

**Dimensionality Reduction:** Technique used to **reduce number of input variables (features)**.

Benefits: Faster computation, Less noise, Better visualization, and Reduced overfitting

Common method: PCA (Principal Component Analysis)

**Principal Component Analysis (PCA):** it transforms correlated variables into **new uncorrelated components** called **principal components**.

Goal: Preserve **maximum variance** with fewer dimensions.

### PCA Concepts

| Term                | Meaning                        |
|---------------------|--------------------------------|
| <b>Eigenvectors</b> | Directions of maximum variance |
| <b>Eigenvalues</b>  | Amount of variance explained   |

Rules:

- **PC1 captures most variance**
- **PC2 captures second largest variance**

### PCA Steps

1. **Standardize data**
2. Compute **covariance matrix**
3. Calculate **eigenvalues and eigenvectors**
4. Select principal components
5. Transform data to new feature space

**Association Rule Learning:** Technique used to **discover relationships between items in large datasets**.

Example: Market Basket Analysis

Example rule: Bread → Butter

Meaning: Customers who buy bread often buy butter.

### Association Rule Metrics:

| Metric            | Meaning                                                                                                                                                      |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Support</b>    | Frequency of itemset in dataset                                                                                                                              |
| <b>Confidence</b> | Probability of Y when X occurs                                                                                                                               |
| <b>Lift</b>       | Strength of association between items <ul style="list-style-type: none"><li>• Lift &gt; 1 → strong association</li><li>• Lift = 1 → no association</li></ul> |

## Lecture 10 – Artificial Neural Networks (ANN) and Backpropagation.

### AI, ML, and Deep Learning

- **Machine Learning (ML):** Branch of AI that uses statistical algorithms to learn from data and make predictions.
- **Deep Learning (DL):** Subfield of ML that uses **artificial neural networks with multiple layers**.

Key difference:

- ML often requires **manual feature engineering**.
- DL **automatically learns features from data**.

### Artificial Neural Network (ANN)

- ANN is a **computational model inspired by the human brain**.
- It consists of **artificial neurons connected by weighted links**.

Purpose: Transform **input data** → **output prediction**.

### Artificial Neuron Concept

A neuron performs computation using:

- **Inputs (x)**
- **Weights (w)**
- **Bias (b)**
- **Activation function**

Output depends on the **weighted sum of inputs**.

General form:  $z = w \cdot x + b$  → Neuron “fires” when the value exceeds a **threshold**.



#### Weights and Bias

- **Weights:** Represent importance of each input.
- **Bias:** Helps shift the decision boundary.

Weighted sum determines neuron output.

**Activation Functions:** determine the **output of a neuron** and introduce **non-linearity**.

Common activation functions:



| Activation Function | Purpose                             |
|---------------------|-------------------------------------|
| Binary Step         | Outputs 0 or 1                      |
| Linear              | Used in regression                  |
| Sigmoid             | Outputs between 0 and 1             |
| Tanh                | Outputs between -1 and 1            |
| ReLU                | Most common in deep learning        |
| Softmax             | Used for multi-class classification |

## Perceptron

- Simplest neural network model.
- Used for **binary classification**.

Key properties:

- Finds a **linear decision boundary**.
- Works only for **linearly separable data**.

Example tasks: AND gate classification.

**Classical / Error correction Perceptron Learning Rule** (Weights are updated when predictions are wrong).

Update formula:

$$w_{new} = w_{old} + \alpha \times output \times input$$

$$w_{new} = w_{old} + \alpha(desired - output) \times input$$

Where:  $\alpha$  = **learning rate**

## Typical Neural Network structure

| Layer           | Role                          |
|-----------------|-------------------------------|
| Input Layer     | Receives input features       |
| Hidden Layer(s) | Extract patterns and features |
| Output Layer    | Produces prediction           |

**Hidden Layers:** allow neural networks to **learn complex patterns**.

Important properties:

- Detect hidden features in data
- Enable modeling of **non-linear relationships**.

With:

- 1 hidden layer → represent continuous functions
- Multiple layers → solve complex problems.

### **Feedforward Neural Network**

- Data flows **only in one direction**:

Input → Hidden Layer → Output.

Characteristics:

- No loops
- Used in basic neural networks.

### **Multilayer Perceptron (MLP)**

Extension of perceptron with **multiple hidden layers**.

Features:

- Fully connected layers
- Uses **gradient descent for learning**
- Supports **forward propagation and backpropagation**.

### **Learning in ANN**

ANN learns by **adjusting weights based on training data**.

Process:

1. Initialize weights randomly.
2. Pass input through network.
3. Compute prediction error.
4. Adjust weights to minimize error.

### **Forward Propagation**

- Input moves **forward through layers** to compute output.

Steps:

1. Compute weighted sums
2. Apply activation functions
3. Produce output prediction.

## Backpropagation

Algorithm used to **update weights by propagating errors backward**.

Process:

1. Perform forward pass.
2. Compute output error.
3. Propagate error backwards through network.
4. Update weights using gradients.

Goal:

- **Minimize total network error.**

## Gradient Descent

Optimization method used to **minimize loss function**.

Key idea:

- Move weights in direction that **reduces error**.

Variants:

- Batch Gradient Descent
- Stochastic Gradient Descent (SGD)
- Mini-batch Gradient Descent

## Training Terminology

| Term      | Meaning                              |
|-----------|--------------------------------------|
| Batch     | Subset of training data              |
| Epoch     | One full pass through entire dataset |
| Iteration | One forward + backward pass          |

Example:

- Dataset: 10,000 samples
- Batch size: 1,000
- Batches per epoch = 10.

*Prof. W. Elashmawi*